RTDL: A Python-Hosted Ray-Tracing DSL for Non-Graphical Spatial Workloads

Rubao Lee
lee.rubao@ieee.org

Abstract—Ray tracing (RT) is widely used for image synthesis because it solves hierarchical geometric search efficiently. The same capability is attractive for non-graphical workloads such as spatial joins, point-in-polygon tests, and geometric candidate generation, but direct use of RT runtimes still requires backend-specific engineering. We present RTDL (Ray-Tracing Domain Language), a Python-hosted DSL and compiler/runtime system for expressing such workloads once and lowering them to multiple backends while preserving explicit semantics. RTDL separates authored kernel meaning from backend-specific execution details such as acceleration-structure construction, launch state, buffer layout, and result materialization. We evaluate RTDL on the RayJoin workload family using stable real-data packages and controlled scalability studies. Across the accepted package surfaces, RTDL preserves exact row-level parity among a native C++ oracle, Embree, OptiX, and an external PostGIS reference. On the main long County–Zipcode point-in-polygon workload reported in this paper, RTDL with Embree and RTDL with OptiX both outperform PostGIS on the published prepared-execution and repeated-call boundaries. The result is a reproducible, multi-backend evaluation showing that a programmable RT language/runtime stack can be a credible way to build non-graphical ray-tracing workloads.

Index Terms—domain-specific language, ray tracing, spatial joins, Embree, OptiX, PostGIS, RayJoin, reproducibility

I. Introduction

The recent proliferation of hardware-accelerated ray-tracing (RT) cores and high-performance traversal kernels has transformed graphics, but their potential for non-graphical spatial computing remains underexploited. Operations such as spatial joins, point-in-polygon tests, and geometric candidate filtering are natural fits for BVH-style traversal and intersection machinery. In practice, however, exploiting that hardware still requires backend-specific engineering around memory layout, launch configuration, payload conventions, and numerical boundary semantics. The programming burden is especially high when the goal is to preserve the same workload semantics across CPU and GPU execution paths.

RTDL is intended to reduce that programming burden by raising the authoring surface from backend-specific APIs to a compact kernel DSL while retaining multiple execution backends. The language is hosted in Python, but its purpose is not scripting convenience alone. Its purpose is to let an author specify geometric inputs, traversal intent, refine predicates, and output rows once, then lower that kernel into controlled runtime paths with comparable semantics. The central question of this paper is whether such an abstraction can remain useful and defensible when evaluated on a demanding, already-published workload family.

RayJoin is the right target for that evaluation because it exercises exactly the class of workloads RTDL is meant to support: non-graphical ray-tracing queries over spatial data with both correctness and performance demands. This paper therefore does two things. First, it presents RTDL as a language/runtime design for this workload class. Second, it evaluates RTDL on a carefully scoped RayJoin-oriented package consisting of stable real-data cases and controlled scaling studies. The scope is intentionally narrower than a full paper-identical reproduction: unavailable dataset families remain deferred, and the current overlay result remains a seed-generation surface rather than full polygon materialization.

This paper makes the following contributions:

1) The RTDL System: We introduce a Python-hosted DSL and compiler that lowers high-level spatial kernels into a backend-independent intermediate representation, separating kernel semantics from backend-specific traversal and scheduling.
2) Multi-Backend Runtime: We describe an extensible runtime architecture that supports execution across Intel Embree (CPU), NVIDIA OptiX (GPU), and a native C++ semantic oracle.
3) Cross-System Correctness Methodology: We establish a rigorous validation framework that ensures exact, row-level parity across heterogeneous execution paths, utilizing PostGIS as an external industrial baseline for real-data packages.
4) RayJoin-Oriented Evaluation: We provide a reproducible, multi-backend evaluation of RayJoin-style LSI, PIP, and overlay-seed workloads on stable datasets, demonstrating that RTDL can preserve exact semantics while targeting heterogeneous RT backends.

II. Related Work

A. Spatial Join Systems

RayJoin is the closest direct comparison point for this paper because RTDL is explicitly evaluated against its workload family, artifact structure, and reporting surfaces [1]. PostGIS remains the most important external reference in the current RTDL evaluation because it provides a widely used industrial spatial database with

indexed spatial predicates and well-understood geometry semantics [2]. GeoSpark SQL represents a different design point: it exposes SQL-level spatial operations on top of a distributed Spark/Hive execution substrate rather than a backend-portable kernel language [3]. Other widely used distributed spatial systems, such as SpatialHadoop and Apache Sedona, occupy the same broad system class even though they expose different storage and execution tradeoffs [4], [5]. RTDL differs from all of these classes of systems. It is not a database, and it is not a paper-specific one-off implementation. Instead, it aims to express non-graphical ray-tracing workloads through a constrained kernel language and then lower them into multiple execution backends while keeping explicit correctness contracts.

B. Compiler and DSL Context

RTDL also relates to compiler-backed domain-specific systems such as Halide [6] and TVM [7]. Those systems separate high-level computation intent from lower-level schedule or backend realization and are therefore important precedents for RTDL's own authored-kernel to backend-plan split. The difference is domain focus. Halide and TVM center image-processing and tensor-operator pipelines, whereas RTDL centers geometric candidate generation, refine predicates, and emitted join rows. The relevant design pressure is therefore not only optimization, but exact spatial semantics across heterogeneous backends.

C. Ray-Tracing Runtime Context

At the runtime layer, RTDL builds on ray-tracing infrastructure rather than reimplementing traversal from scratch. Embree provides the validated CPU ray-tracing kernel framework [8], while OptiX provides the reported GPU execution path in this paper [9]. RTDL's contribution is not a new traversal engine. The contribution is the language/runtime boundary, the native oracle, and the reproducible multi-system evaluation built on top of these runtimes.

III. What Ray Tracing Is and Why It Matters Beyond Graphics

In graphics, ray tracing answers visibility questions by traversing geometric hierarchies quickly. Rays, bounding-volume hierarchies, and fast intersection kernels make it practical to search complex geometric scenes without checking every primitive pair directly. The same basic pattern also appears in non-graphical workloads. A spatial join, for example, can often be expressed as a broad-phase search for candidate geometry pairs followed by an exact refine step on the surviving candidates.

This observation is what makes ray tracing interesting for spatial computing. Workloads such as line-segment intersection, point-in-polygon, overlap candidate generation, and geometric counting all involve the same kind of hierarchical search that RT systems already accelerate well. Systems such as RayJoin [1] demonstrate that this mapping is not merely conceptual: ray-tracing cores can be used effectively for spatial join work.

The problem is that using RT backends directly still exposes a great deal of backend-specific engineering. Developers must reason about acceleration structures, payload/state conventions, memory layout, launch details, and boundary behavior separately for CPU and GPU paths. RTDL addresses that problem instead of trying to replace the underlying traversal engines.

A. RayJoin as the First Application Target

RayJoin is the right first target for RTDL because it exercises the exact class of workloads RTDL is meant to support [1].

The current RTDL paper does not claim a paper-identical reproduction of every RayJoin dataset family. Instead, it uses RayJoin as the workload and evaluation reference point for a more language/runtime-oriented question: can one authored kernel surface survive lowering to multiple RT backends while preserving exact semantics and producing honest performance claims?

The evaluation surface mirrored from RayJoin includes three workload families:

- LSI: line-segment intersection style joins,
- PIP: point-in-polygon style joins,
- Overlay: currently represented in RTDL as an overlay-seed evaluation, which identifies intersecting polygon pairs without full geometry materialization.

For this work, RayJoin Figure 13 and Figure 14 are represented through controlled synthetic studies, while RayJoin Table 3, Table 4, and Figure 15 are represented through validated results on stable dataset packages.

IV. Why RTDL and How It Works

A. Why a DSL Is Needed

The central design claim of RTDL is that non-graphical RT workloads should be programmable at the workload level rather than at the backend-API level. Without such an abstraction, every new workload must be translated manually into backend-specific setup code, traversal configuration, payload plumbing, buffer layout, and result marshaling. That makes correctness difficult to compare across systems and slows down experimentation.

RTDL therefore tries to keep the authored surface close to the workload itself:

- which geometries are inputs,
- which side is probe or build,
- what traversal is intended,
- what refine predicate defines final truth,
- which fields belong in the emitted rows.

The backend-specific execution details are still present, but they move below the language boundary instead of staying in every authored workload.

B. Language Surface

RTDL is authored from Python through a constrained kernel DSL. A user writes a kernel by declaring typed inputs, traversal intent, refine predicates, and output fields. The intent is that authors specify workload meaning rather than backend launch details. The same language surface currently supports six implemented workloads in the repository, but the RayJoin-facing slice of this paper focuses on LSI, PIP, and overlay.

C. Representative Kernel

Listing 1 shows the shape of an RTDL PIP kernel. The author declares probe/build inputs, asks for BVH traversal, selects a refine predicate, and specifies the emitted row schema. Backend-specific details such as Embree scene construction, OptiX launch state, and row marshaling remain outside the authored kernel.

Listing 1. Representative RTDL PIP kernel.

```
@rt.kernel(backend="rtdl", precision="float_approx")
def point_in_counties():
    points = rt.input("points", rt.Points, role="probe")
    polygons = rt.input("polygons", rt.Polygons, role="build")
    candidates = rt.traverse(points, polygons, accel="bvh")
    hits = rt.refine(
        candidates,
        predicate=rt.point_in_polygon(exact=False,
                                      boundary_mode="inclusive"),
    )
    return rt.emit(hits, fields=["point_id", "polygon_id", "contains"])
```

D. Compiled Representation and Lowering

RTDL lowers authored kernels into a compiled-kernel representation that separates language semantics from backend realization. The lowering path remains RayJoin-oriented in the current codebase: it maps geometry roles, candidate generation, refine predicates, and emitted rows into backend-specific execution plans. In practice, the lowered form records the probe/build role assignment, the traversal primitive to invoke, the refine predicate family, precision and boundary-mode settings, and the emitted row schema. The backend adapters then translate that representation into concrete execution state such as Embree scenes, OptiX launch parameters, native buffers, and row marshaling logic. This allows the same kernel to execute against a semantic oracle, a CPU Embree runtime, and a GPU OptiX runtime without changing the authored kernel surface.

E. Execution Backends

The evaluated backends for this paper are:

- Native C/C++ oracle: the semantic ground truth for the reported results.
- Embree: the primary validated CPU backend.
- OptiX: the validated GPU backend on the Linux workstation in Table I.

The Vulkan backend remains in the repository, but the paper's main performance story focuses on Embree and OptiX because those are the mature high-performance backends on the accepted long exact-source surface.

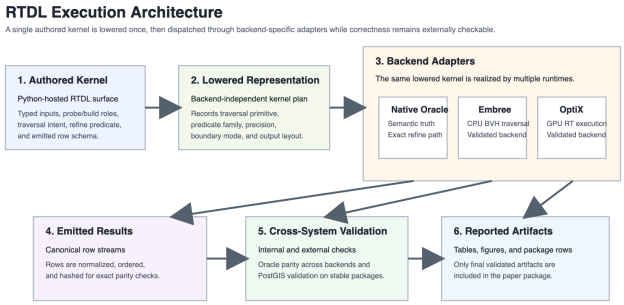

Fig. 1. RTDL execution architecture. A single authored kernel is lowered into a backend-independent representation, dispatched through backend adapters, and checked through emitted-row parity and external validation.

F. Execution Pipeline

The execution path is conceptually simple even though the backend implementations are not. RTDL first compiles the authored kernel into a backend-independent representation that records geometry roles, traversal structure, refine predicate, and emitted fields. The runtime then normalizes input geometry, invokes the selected backend, and converts the returned native rows into the requested user-facing representation. The crucial discipline in this paper is that all reported real-data results are checked against the native oracle and, for the validated real-data packages, against PostGIS as an external reference.

This separation between authored kernel, lowered representation, and runtime adapter is the main systems contribution of RTDL. It keeps semantic decisions visible at the language level while still allowing backend-specific execution strategies underneath. The resulting paper is therefore not just a benchmark of Embree or OptiX; it is an evaluation of whether one authored workload surface can survive translation into multiple RT backends without drifting semantically.

V. Design Considerations and Evaluation Methodology

A. Design Considerations

Three design considerations drive the current RTDL package.

First, semantic consistency is more important than backend-local cleverness. The paper compares systems

that execute related geometric work under different runtime conditions, so the language/runtime layer must make boundary modes, refine predicates, and emitted rows explicit enough to compare honestly.

Second, RTDL must separate broad-phase candidate generation from final truth. That principle shaped the mature positive-hit point-in-polygon paths reported here. Embree and OptiX both use backend-specific candidate generation, but the reported accepted paths keep final inclusive truth on the host through exact finalization.

Third, the Python host is allowed to orchestrate but should not repeatedly do bulk geometry work on the hot path. The strongest current performance wins only appeared after the runtime began reusing prepared executions and prepacked geometry instead of rebuilding them unnecessarily.

B. Optimization Strategy

The optimization story in this paper is therefore not "replace PostGIS with a single GPU kernel." It is a layered systems story.

At the backend level, RTDL relies on mature traversal runtimes such as Embree and OptiX. At the language/runtime level, the main optimizations are:

- over-approximating backend candidate generation with host exact finalize,
- prepared-execution reuse,
- canonical-record and cached-prepared fast paths,
- reuse of prepacked point and polygon buffers,
- moving one-time launch costs out of repeated timed prepared execution surfaces when that boundary already charges them to preparation.

These optimizations matter because the practical performance gap in a DSL system often comes not from raw traversal speed, but from repeated packing, binding, and result-materialization overhead around the backend itself.

C. Correctness Method

The correctness methodology is intentionally layered. The native C/C++ oracle serves as the internal semantic ground truth. Embree and OptiX are checked against that oracle. For validated real-data packages, PostGIS serves as an external industrial reference. PostGIS queries are executed with indexed GiST-assisted plans rather than brute-force SQL. For PIP, PostGIS positive-hit queries are expanded into the same full-matrix RTDL truth semantics before parity comparison.

The PIP parity path used in this paper is also more specific than a generic manual point-in-polygon implementation. During development, a systematic external comparison against PostGIS exposed remaining boundary/-topology mismatches, so the RTDL refine path adopted GEOS-backed prepared-polygon covers(point) semantics for the native oracle and Embree, and the reported OptiX overlay result was observed on a GEOS-enabled build on the validated Linux host [10]. This detail matters because the paper claims exact parity with PostGIS on the validated packages, not merely approximate geometric agreement. This does align RTDL's final refine step more closely with the geometric engine family used by PostGIS, but that alignment is intentional: the goal is to remove semantic ambiguity in the final truth predicate, not to erase backend differences in candidate generation.

TABLE I
Execution environment for the reported four-system package.

Item	Value
Host class	Linux validation workstation
OS	Ubuntu 24.04.4 LTS
CPU	Intel Core i7-7700HQ
Memory	16 GiB installed (about 15 GiB usable)
GPU	NVIDIA GeForce GTX 1070
Driver	580.126.09
CPU RT backend	Embree 4.3.0
GPU RT backend	OptiX 9.0.0
External checker	local PostgreSQL/PostGIS on the same host
GEOS role	exact prepared-polygon covers(point) path for validated PIP parity

Parity is checked at the emitted-row level. In the validated artifacts, each system produces a canonical row order, and the final row stream is hashed with SHA-256. This paper therefore includes only cases whose final reported artifacts established exact hash-level agreement across the compared systems.

D. Experimental Scope

This paper uses only final validated artifacts from the repository history. A case is included only if it is already validated and clearly scoped. The paper therefore distinguishes:

- validated package: a stable real-data package or synthetic evaluation surface with completed parity and timing evidence,
- deferred-unavailable: excluded because the public acquisition path is unstable or unavailable,
- not claimed: paper-identical or nationwide statements that the current evidence does not justify.

All reported timings and parity statements in this paper come from the final validated artifacts in the repository history. Earlier provisional or superseded measurements are intentionally excluded.

E. Experimental Environment

Table I summarizes the host environment for the reported four-system results. All four-system runs reported in this paper were executed on the same Linux validation workstation.

F. Packages Used In This Paper

The current manuscript is grounded in one bounded package family and one stronger flagship evaluation sur-

face. The bounded package includes County ⋈ Zipcode on top4_tx_ca_ny_pa, BlockGroup ⋈ WaterBodies on county2300_s10, and a bounded Australia LKAU ⋈ PKAU package. The strongest current flagship surface is the long County ⋈ Zipcode positive-hit PIP evaluation used throughout the later RTDL backend-closure work. These packages were all validated on the Linux validation workstation, and the bounded Australia package was staged from live OSM Overpass way geometry and then frozen as the validated Australia slice [11].

G. Overlay-Seed Contract

RTDL's current overlay result is not full polygon materialization. The reported surface is an overlay-seed evaluation that records candidate pair information and whether later LSI or PIP refine is required. This is the current semantics-preserving surface compared against PostGIS in the validated overlay package, and it is intentionally narrower than the full overlay output reported in the original RayJoin paper.

VI. Current Evaluation Status

The purpose of the current paper draft is to establish that RTDL is real, programmable, and already strong enough to support meaningful non-graphical ray-tracing workloads across multiple backends. The detailed experimental artifact trail remains in the repository reports rather than in the paper body. The current accepted result can therefore be stated compactly. RTDL preserves exact emitted-row parity across the validated packages and uses PostGIS as the external indexed reference on the main real-data surfaces. On the strongest current long County ⋈ Zipcode positive-hit PIP surface, Embree and OptiX both beat PostGIS on the accepted prepared and repeated-call boundaries. Vulkan is hardware-validated and parity-clean on that same surface, but slower. That is enough for the paper's present purpose: to show that a Python-hosted ray-tracing DSL/runtime stack is feasible, correct on the accepted package surfaces, and already capable of competitive backend execution on the flagship workload family.

VII. Relationship to RayJoin

RayJoin provides the primary workload and evaluation reference for this paper. RTDL does not attempt to replace RayJoin's contribution; instead, it asks a different question: can the same workload family be expressed through a programmable DSL/runtime surface while preserving exact semantics across multiple execution backends? The answer reported here is deliberately scoped. We mirror RayJoin's major workload categories and evaluation surfaces, but we restrict the paper to stable, reproducible packages whose results can be checked both against an internal oracle and, where appropriate, against PostGIS. Unavailable or unstable dataset families are deferred rather than used to support stronger reproduction claims than the evidence allows.

A. Workload Surface

The RTDL evaluation in this paper covers the same workload family categories that matter to RayJoin: LSI, PIP, and overlay composition.

B. Artifact Structure

The reported evaluation covers:

- Table 3-style package rows,
- Figure 13-style LSI scalability studies,
- Figure 14-style PIP scalability studies,
- Table 4-style overlay-seed rows,
- Figure 15-style overlay-seed speedup comparisons.

C. Key Differences from the Original Paper

The current RTDL evaluation differs from the original RayJoin paper in four main ways:

1) It is package-scale rather than paper-scale.
2) It relies on controlled synthetic studies where exact paper data is unavailable.
3) It uses PostGIS as an external correctness reference, which RayJoin did not use in the same way.
4) The current overlay surface is a seed-generation study, not full polygon overlay materialization.

VIII. Limitations

This work is intentionally explicit about its limitations.

First, this is not a paper-identical reproduction. Several continent-scale lakes/parks families remain deferred because their public acquisition paths are unstable or unavailable. In the current evaluation these deferred families are:

- LKAF ⋈ PKAF,
- LKAS ⋈ PKAS,
- LKEU ⋈ PKEU,
- LKNA ⋈ PKNA,
- LKSA ⋈ PKSA.

Second, the practical substitute for the paper's Block ⋈ Water row is the county2300_s10 package over BlockGroup and WaterBodies. This scoping qualifier is material to interpretation.

Third, only LKAU ⋈ PKAU currently has four-system overlay-seed results. County/Zipcode and BlockGroup/WaterBodies still provide useful overlay context from earlier Embree work, but not the same four-system overlay evidence.

Fourth, Vulkan is not part of the paper's main performance claim. It is included only as a parity-clean but slower supporting backend on the accepted long exact-source surface.

IX. Conclusion

RTDL now supports an externally checked, multi-backend evaluation of the RayJoin workload family that is substantial enough to analyze in paper form. The contribution of this work is not a claim of full paper-identical reproduction. It is a technically grounded evaluation based

on stable real-data results, controlled scaling studies, external PostGIS checks, and exact row-level agreement across the reported systems. On the accepted long exact-source County ⋈ Zipcode positive-hit PIP surface, Embree and OptiX now outperform PostGIS on the reported prepared and repeated-call boundaries, while Vulkan remains parity-clean but slower. Across the bounded package and the long exact-source backend surface, the reported rows preserve exact hash-level emitted-row agreement. That evaluation provides a defensible baseline for subsequent workload expansion, including deferred dataset families, stronger Vulkan maturation, and fuller overlay materialization, without overstating what the current evidence proves.

References

[1] L. Geng, R. Lee, and X. Zhang, "Rayjoin: Fast and precise spatial join," in Proceedings of the 38th ACM International Conference on Supercomputing, ser. ICS '24. ACM, 2024. [Online]. Available: https://doi.org/10.1145/3650200.3656610

[2] PostGIS Project Steering Committee, "Postgis manual," https://postgis.net/documentation/, 2026, used as the external spatial database reference in this work.

[3] Z. Huang, Y. Chen, L. Wan, and X. Peng, "Geospark sql: An effective framework enabling spatial queries on spark," ISPRS International Journal of Geo-Information, vol. 6, no. 9, p. 285, 2017. [Online]. Available: https://www.mdpi.com/2220-9964/6/9/285

[4] SpatialHadoop Project, "Spatialhadoop," http://spatialhadoop.cs.umn.edu/, 2026, original public catalog referenced by the deferred lakes/parks continent families.

[5] Apache Software Foundation, "Apache sedona," https://sedona.apache.org/, 2026, representative modern distributed spatial analytics system.

[6] J. Ragan-Kelley, C. Barnes, A. Adams, S. Paris, F. Durand, and S. Amarasinghe, "Halide: A language and compiler for optimizing parallelism, locality, and recomputation in image processing pipelines," in Proceedings of the 34th ACM SIGPLAN Conference on Programming Language Design and Implementation, ser. PLDI '13. ACM, 2013, pp. 519–530. [Online]. Available: https://people.csail.mit.edu/jrk/halide-pldi13.pdf

[7] T. Chen, T. Moreau, Z. Jiang, L. Zheng, E. Yan, H. Shen, M. Cowan, L. Wang, Y. Hu, L. Ceze, C. Guestrin, and A. Krishnamurthy, "TVM: An automated end-to-end optimizing compiler for deep learning," in 13th USENIX Symposium on Operating Systems Design and Implementation, ser. OSDI '18, 2018, pp. 578–594. [Online]. Available: https://www.usenix.org/conference/osdi18/presentation/chen

[8] I. Wald, S. Woop, C. Benthin, G. S. Johnson, and M. Ernst, "Embree: A kernel framework for efficient cpu ray tracing," ACM Transactions on Graphics, vol. 33, no. 4, pp. 143:1–143:8, 2014.

[9] S. G. Parker, J. Bigler, A. Dietrich, H. Friedrich, J. Hoberock, D. Luebke, D. McAllister, M. McGuire, K. Morley, A. Robison, and M. Stich, "Optix: A general purpose ray tracing engine," in ACM SIGGRAPH 2010 Papers, ser. SIGGRAPH '10. ACM, 2010, pp. 66:1–66:13. [Online]. Available: https://research.nvidia.com/publication/2010-08_optix-general-purpose-ray-tracing-engine

[10] GEOS Project, "Geos c api documentation," https://libgeos.org/doxygen/c_iface.html, 2026, referenced for the prepared-geometry exact point-in-polygon path used in accepted RTDL parity runs.

[11] OpenStreetMap Foundation, "Overpass api," https://overpass-api.de/, 2026, used for the bounded Australia lakes/parks source acquisition path.